WSL 部署完成后的项目改造指南

ScholarHub 大数据毕业设计 · 实操解释版 · 2026-10-17

1．你现在做到哪里，接下来要做成什么

下一步的目标：把 ScholarHub 的学习记录交给 Spark 计算，再把计算结果显示在项目里。先完成一条能演示、能核对的完整流程。

部分	当前状态	后续工作
Vue 页面、FastAPI 接口、MySQL	已有项目代码	继续运行现有功能，复用认证与图表组件
WSL2、Ubuntu 24.04、Java 17	已安装并核实	作为数据分析运行环境
Ubuntu Python 3.12、PySpark 4.2.0	独立环境中已核实	使用现有环境编写分析程序
Spark 最小计算	你已得到 Row count: 10	继续验证文件读写、清洗和聚合
项目数据进入 Spark、结果回到网页	当前尚未实现	本文的实际改造任务

当前代码基线为 main 的 9320d68 及现有工作区。原统计接口使用 SQLAlchemy 查询 MySQL；原 AI 学习报告收集业务事实后调用模型。这两项现有能力继续保留，新分析有独立的数据来源标识。

最终要跑通的流程

```
Windows：MySQL 学习记录、每日任务
        ↓ 导出一份固定数据文件
Ubuntu：Spark 清洗 → Spark SQL 统计 → 保存结果
        ↓ 把汇总结果交回 Windows
Windows：写入分析结果表 → FastAPI 查询 → Vue 图表
```

你最终能在“本人离线分析”页面看到：已记录学习分钟、有记录的天数、任务完成率，以及数据截至时间。网页读取已经算好的结果；点击图表时不启动 Spark。

本文的阅读方式

先看第 2、3 节理解运行位置和计算内容，再按第 4 至 8 节开发，最后用第 9 节检查进度。本文是具体执行指南，补充 10 月 16 日总体方案；下文文件、表和接口中标注“拟新增”的内容均待实现。

2．Windows 和 WSL 分别做什么

位置	负责的事情	如何理解
Windows	Vue、FastAPI、MySQL、项目 Git 仓库	继续运行你现在的系统
Ubuntu	PySpark 程序和分析数据文件	为原项目增加一个计算工具
Windows 的 VS Code	编辑原项目和新增脚本	通过 WSL 扩展调试 Linux 程序

代码一份，运行环境分开

建议在现有仓库新增 analytics/ 保存分析源码；Ubuntu 已建的 ~/scholarhub-analytics/ 保存虚拟环境和运行数据。Linux 中只运行新增分析脚本，避免再维护一份完整的前后端副本。

```
ScholarHub/                 现有 Git 仓库
  frontend/                 原前端 + 拟新增分析页面
  backend/                  原后端 + 拟新增导出、导入和接口
  analytics/                拟新增 Spark 源码、小型演示样本

~/scholarhub-analytics/           Ubuntu 运行目录
  .venv/                    已有的 PySpark 虚拟环境
  data/                     拟新增原始、清洗和汇总数据
  experiments/              拟新增规模实验结果
```

大量数据和 Parquet 文件放在 Linux 的 data/ 下，便于 Linux 工具读写；源码仍统一由原仓库管理。微软建议将频繁处理的数据放在运行工具所属的文件系统中。WSL 文件存储说明

两边如何交换数据

先约定一个本地交换目录，例如 C:\scholarhub-data\exchange；Ubuntu 通过 /mnt/c/scholarhub-data/exchange 访问同一位置。Windows 导出文件，Ubuntu 复制到 Linux 数据目录计算；计算完成后，仅把小型汇总文件复制回来，由 Windows 后端导入 MySQL。目录由配置或命令参数传入。

当前可执行的环境切换命令，在 Ubuntu 中运行：

```
cd ~/scholarhub-analytics
source .venv/bin/activate
```

第一版采用文件交换，继续使用现有 MySQL 连接方式。WSL 直接连接 Windows 数据库、定时调度和一键启动可以在手动流程稳定后再评估。

3．先确定要算的三个结果

下面全部是虚构的小样本，只用于解释规则。它们代表用户 A 的学习记录，来源判断以真实反馈关联为准。

记录 ID	北京时间日期	记录分钟	来源说明
101	10 月 15 日	30	普通记录
102	10 月 15 日	45	普通记录
103	10 月 16 日	NULL	有 ACCEPT 反馈关联
103	10 月 16 日	NULL	重复导入的同一条记录
104	10 月 16 日	0	历史零分钟边界样例

同一 ID、内容也相同的重复记录只保留一条。因此原始 5 行变成有效 4 条，总分钟为 30 + 45 + 0 = 75，时长缺失 1 条，有记录的日期为 2 天。0 是已知零值，NULL 是未记录时长，二者分别统计。

指标	计算方法	这个例子的答案
已记录学习分钟	累加非 NULL 的 duration_minutes	75 分钟，同时说明缺失 1 条
记录活跃天数与来源	对存在有效记录的日期去重；拆分普通与可信教学来源	2 天；普通 3 条、可信教学 1 条
按计划日期任务完成率	已完成任务数 ÷ 该日期范围内全部保留任务数	4 个任务完成 2 个，即 50%

任务例子为：已完成 2 个、进行中 1 个、已取消 1 个。首版分母包含取消任务，与当前学习报告的计数规则保持一致；页面同时显示取消数。周期汇总使用总完成数除以总任务数，不平均每天的百分比。

有四个容易算错的地方

- 时长来自用户记录，页面名称使用“已记录学习分钟”，不称作自动测得的专注时长；已有新建接口的校验规则继续保留。
- 没有任务时，完成率显示“无样本”；有任务但完成 0 个时，显示 0%。
- 199 个完成、200 个任务是 99.5%，不能用四舍五入后的 100 判断全部完成。
- 记录按 occurred_at 转北京时间确定日期；任务直接使用 scheduled_date，它原本就是日期，不再做时区转换。

首版固定 Asia/Shanghai 业务日并标注在页面上，避免同一个批次被不同浏览器时区解释成不同结果。

4．第一、二阶段：先跑小样本，再接入项目数据

第一阶段：让 Spark 读取一个文件并给出正确答案

先只做学习记录分钟汇总。新增少量虚构样本，覆盖正常值、NULL、0、同 ID 重复和跨午夜；同时保存能手算的期望结果。此时不需要数据库迁移或前端页面。

拟新增位置	作用
analytics/requirements.txt	记录已选的 PySpark 4.2.0 分析依赖
analytics/fixtures/	保存小型虚构样本及期望结果
analytics/jobs/run_batch.py	命令行入口，接收输入、输出和批次参数
analytics/jobs/clean_records.py	校验学习记录并生成清洗后明细
analytics/jobs/aggregate_daily.py	按用户、日期计算分钟和记录数量
analytics/tests/	用已知答案验证清洗和汇总

先让一条入口完成读文件、清洗、汇总、写文件；只在职责明确后拆分上述模块。验收依据是输出文件的内容与手算结果一致，以及 Python 工作进程、Parquet 写入和重新读取都成功。

第二阶段：从现有 MySQL 导出固定快照

由 Windows 后端的命令行脚本执行导出，复用已有 SQLAlchemy 和 PyMySQL。第一批只选明确授权的演示账号和一个日期范围。

数据来源	需要导出的字段
learning_records	id、user_id、project_id、task_id、record_type、duration_minutes、occurred_at、created_at
daily_tasks	id、user_id、status、scheduled_date、completed_at
feedback	与已选记录关联的 learning_record_id、decision

采用字段白名单，不导出密码、令牌、记录正文和教师评语。以同一个只读一致性事务读取这些表，写入 UTF-8 数据文件；随批次记录抽取时间、日期范围、时区、字段版本和各表行数。

建议新增 backend/app/cli/export_analytics.py，将查询细节放入后端 Repository。先沿用显式字段的 JSON Lines 文件，NULL 保留为 JSON null，时间使用带 Z 的 UTC 字符串；CSV 只作为第一阶段练习输入。

learning_records 可以修改和删除，却没有 updated_at，第一版按选定日期窗口完整导出。它代表抽取时刻的快照，不能只追加最大 ID 之后的数据，也不能复原任意过去时刻。

验收：文件行数和已选账号、日期范围可核对；不写业务表；同一份快照能反复送入 Spark。以上入口均为拟新增，尚无可直接执行的导出命令。

5．第三阶段：写出真正处理项目数据的 Spark 程序

三个名称分别代表什么

名称	简单解释	保存什么
ODS 原始层	从数据库取出的原样备份	固定批次的原始文件和说明
DWD 明细层	校验、去重、统一日期后的明细	清洗后的记录、任务和异常原因
ADS 汇总层	网页真正需要的统计结果	按用户和日期计算的数字

这些是目录和数据职责划分，首版用本地文件即可。DWD、ADS 建议使用 Parquet，Spark 支持保留字段结构的读取和写入。Spark Parquet 文档

```
data/<数据集>/<批次>/
  ods/       原始记录、任务、反馈关联、批次说明
  dwd/       清洗后明细、异常记录
  ads/       每用户每日汇总、质量计数
```

程序按这个顺序处理

1. 读取批次说明，以明确的 schema 解析字段，拒绝不支持的版本。
2. 校验编号、枚举、时间和分钟值。历史 0 保留；NULL 保留缺失语义；负值、非法日期和非法枚举进入异常文件。
3. 对相同 ID、相同内容的重复行去重；若同 ID 出现冲突内容，报告失败，不随机选一条。
4. 学习记录与 feedback 按 learning_record_id 关联。只有真实关联且 decision 为 accept 才归为可信教学来源，不能相信 JSON 中自写的 source。
5. occurred_at 从 UTC 转北京时间形成 biz_date；再次核对日期窗口和抽取范围。
6. 记录与任务分别聚合，再按 user_id、biz_date 合并日汇总，覆盖“只有记录”或“只有任务”的日期。避免直接把两张明细表相连造成重复累加。
7. 输出 DWD、ADS 和质量计数。DataFrame 做清洗，Spark SQL 做汇总；统计结果写入成功后才进入后续导入。

如何知道它算对了

核对原始行数、重复数、异常数、有效数之间的关系；不同异常原因可多标签，但用于对账的异常行数每行只计一次。时长缺失是有效记录上的单独计数，不当作异常行重复扣减。

同一快照、同一规则重复计算，应得到相同的业务结果。对比数据内容，不比较 Parquet 文件的字节、文件名或行顺序。处理使用 Spark 分布式算子，不能把完整大表 collect 到 Python 后再统计；仅收集有界的验收结果。
DataFrame 说明

6．第四阶段：把结果保存回 MySQL

Spark 先写结果文件。Windows 后端读取小型日汇总，验证后写入 MySQL。这样继续复用现有数据库驱动；Spark 环境只负责计算。

首版拟新增两张分析表

表	保存内容	为什么需要
analytics_batches	批次 ID、owner_user_id、数据集类型、统计起止日、抽取截止时间、时区、规则版本、质量计数、状态、完成时间	知道这批结果归谁、从哪里来、能否展示
analytics_learning_daily	批次 ID、用户 ID、业务日、记录数、分钟、缺失数、来源计数、任务总数和各状态数	为本人页面提供日趋势和周期汇总

首版每批仅对应一个明确授权用户，owner_user_id 记录其归属，即使结果为空也能鉴权。日汇总以 batch_id、user_id、biz_date 唯一，关联批次及合法用户；导入时核对 user_id 与批次归属一致。保存完成数和总数，完成率由确定性规则计算；无任务返回 null。

发布一批结果的过程

```
新建 loading 批次
  → 校验结果文件、用户范围、唯一键和质量计数
  → 分批导入该批次的日汇总
  → 短事务内核对并标记 completed
  → 查询接口开始读取这批结果
```

只有 completed 批次可以展示。导入中途失败时标记 failed，网页继续读取上一成功批次并显示截止时间；只有未发布批次允许清理自身暂存行后重试。已完成批次不能被覆盖，源记录发生修改或规则改变时创建新批次。

批次 ID 必须同时出现在结果文件、导入记录、API 返回和页面说明里。多个图表固定读取同一个批次，避免一次页面展示混入不同次计算。

数据库改造的落地顺序

先提交两张表的字段、约束、索引和增量迁移设计，审查通过后在明确指定的专用测试库验证，再安排正式环境迁移。拟新增 backend/app/models/analytics.py、对应 Repository，以及 backend/app/cli/import_analytics.py。

测试中验证：重复导入不会产生重复日汇总；非法用户、错误批次或失败文件不能发布；同一批次并发导入必须被状态校验和唯一约束阻止。恢复时先停止新批次发布，页面可回到上一成功批次；删除新表属于单独审查的操作。

合成实验只在文件目录中计算，不把虚构用户编号导入业务库。大规模明细保留在 Linux 文件层，MySQL 只保存网页需要的汇总。

7．第五阶段：新增接口和本人分析页面

后端：两个拟新增的只读接口

拟新增接口	参数与返回内容
GET /api/v1/offline-analytics/me/latest	输入日期范围；返回本人可用且覆盖该范围的最新成功批次说明
GET /api/v1/offline-analytics/me/daily	输入 batch_id 和日期范围；返回本人日明细、周期汇总及同批次说明

日期默认最近 7 个完整业务日，单次最多 90 天；只选择覆盖所选日期、时区与规则匹配、owner_user_id 为当前用户的批次。latest 没有可用批次时返回明确的无结果状态；daily 请求不存在或无权访问的批次时返回统一的未找到错误，错误结构沿用现有项目约定。

user_id 从登录身份取得，不接受前端任意传入其他用户 ID。教师身份本身不赋予读取全部个人学习记录的权限。本人学习分析先完成，班级分析作为独立扩展设计。

接口分层沿用 Router → Service → Repository；拟新增 api/v1/offline_analytics.py、services/offline_analytics.py、repositories/analytics.py、schemas/offline_analytics.py，在 api/v1/router.py 和 api/deps.py 接入。原 /statistics 和 /learning-reports 接口继续保留。

前端：一张新页面复用现有风格

拟新增 /offline-analytics 路由和 OfflineAnalyticsView.vue。页面包含日期范围、三项核心结果、分钟趋势、任务完成趋势，以及批次、截止时间、时区、规则版本和缺失计数说明。

文件或位置	计划变化
frontend/src/api/offlineAnalytics.ts	使用既有 apiClient 请求新增接口
frontend/src/types/offlineAnalytics.ts	定义响应、批次和日汇总类型
frontend/src/views/OfflineAnalyticsView.vue	组合查询状态与图表
frontend/src/router/index.ts	添加新路由
frontend/src/layouts/AppLayout.vue	增加本人离线分析入口
frontend/src/components/EChartPanel.vue	优先复用已有图表封装

页面先查询 latest，再固定 batch_id 获取 daily。日期改变或切换账号后，取消或忽略旧响应；退出时清除本人结果。当前只服务一个页面，状态可放在页面或局部 composable 中，确有跨页共享时再使用 Store。

验收覆盖：加载、没有批次、周期内没有记录、0 值、NULL 完成率、请求失败、旧批次提示、账号切换，以及桌面和手机布局。曲线中的空日期可补零，但必须区分“该日无记录”和“整个周期尚未完成分析”。

8．第六阶段：实验、回归和毕业答辩

先验证答案，再记录性能

检查	怎么做	留下什么证据
规则正确	固定样本覆盖重复、NULL、0、跨日、取消、无任务和来源关系	输入、期望结果、实际结果
全链路	一份快照经 Spark、汇总导入、API 到图表	同一 batch_id 的文件、返回样例和截图
权限与失败	A 不能读 B；切换账号不泄漏；导入失败不发布	对应测试结果和错误状态
原功能回归	按变更影响检查登录、学习记录、任务、原统计和 AI 报告	相关测试、前端类型检查和构建结果
规模实验	先 1 万，再 10 万；资源允许再尝试 100 万行	真实行数、配置、每次耗时和输出

规模数据使用固定种子和明确生成规则，单独标注 synthetic。它用于验证程序在不同输入规模下的表现；真实用户数量和教学效果需要另外的事实支持。

基线可以使用 Python 标准库读取同一份文件并计算相同指标。先核对输出一致，再比较耗时；区分 Spark 启动、读取、清洗、聚合和端到端时间。规模固定后各测至少三次，保留原始值和中位数，也记录失败或内存不足。

16GB 电脑先使用 local[2] 和有界数据规模。论文写明“WSL2 下的单机本地多线程实验”；这能验证 Spark 程序与数据处理流程，不能当作多台机器的集群性能。Spark 小样本耗时可能较长，按实际结果解释启动和调度开销。

答辩按这一条故事线演示

1. 展示演示账号的一条学习记录和一个任务。
2. 展示固定快照、时间范围和字段说明。
3. 运行 Spark，解释去重、时区和可信来源处理。
4. 展示日汇总及质量计数，核对一个手算答案。
5. 发布成功批次，打开本人分析页面。
6. 指出 API、图表和文件中的相同批次 ID，再展示规模实验记录。

你需要能解释：为什么不让网页现场跑 Spark；为什么 NULL 不能直接当成已知 0；为什么任务先单独聚合；为什么源数据改动后要生成新批次。AI 可以帮助解释和开发，最终指标由可复现的确定性程序计算。

这一轮主线完成后，再根据导师要求选择推荐算法、班级汇总、AI 分步实践、HDFS 或实时计算。它们分别增加数据、权限或运维成本，当前先将上述链路做完整。

9．接下来按什么顺序推进

顺序	这一步完成的东西	达到后再继续
1	文件到 Spark 再到结果文件的小样本链路	能手算，能写入并重读 Parquet
2	Windows 只读导出项目快照	来源、账号、日期、行数和字段可核对
3	完整三指标及质量报告	来源、NULL、时区和任务分母正确
4	两张分析表与批次导入发布	失败不展示，重试不重复，成功结果可追踪
5	本人只读 API 和分析页面	一份数据全链路一致，越权被拒绝
6	实验、回归与答辩材料	有真实原始记录，能独立解释流程

下一次可以直接给开发助手的任务

先实施本指南第一阶段：在现有 ScholarHub 仓库中新增最小 analytics 模块，复用 Ubuntu 已安装的 PySpark 4.2.0。使用虚构小样本实现学习记录读取、显式 schema、去重、分钟及缺失数统计、Parquet 写入与重读，并保存手算期望值及验证结果。先检查现状和最小设计，不修改现有接口、数据库或前端；不安装额外依赖。源码只保存一份，运行数据留在 WSL，完成后解释每个文件的作用。

阅读现有代码时，先看这些入口

- 学习记录与任务模型：理解真实字段、日期和关联。
- 反馈模型：理解可信教学来源的依据。
- 当前统计接口 与 查询实现：理解已有 SQL 统计。
- 现有学习报告取数：核对时长、活跃日和任务分母。
- 当前分析页面 与 图表封装：复用页面结构和图表能力。

表结构、迁移和公共接口的具体设计，在相应阶段先审查再实施。本文只生成改造说明，未创建上述业务功能或执行数据库迁移。原总体方案保留供查阅，日常推进以本指南的阶段验收为入口。

参考资料：PySpark 安装、DataFrame 入门、Parquet 读写、WSL 文件系统。核对日期：2026-10-17。